

외국어 노래 속 문장을 듣고 → 따라읽거나 받아적고 → 평가를 확인하는 학습 도우미

https://songlingo.kr

강은영 (https://github.com/greenkey20/song-lingo)

프로젝트 목표

문제: 외국어 노래를 듣다가 무슨 언어인지, 무슨 뜻인지, 어떻게 발음하고 표기하는지 궁금하다!

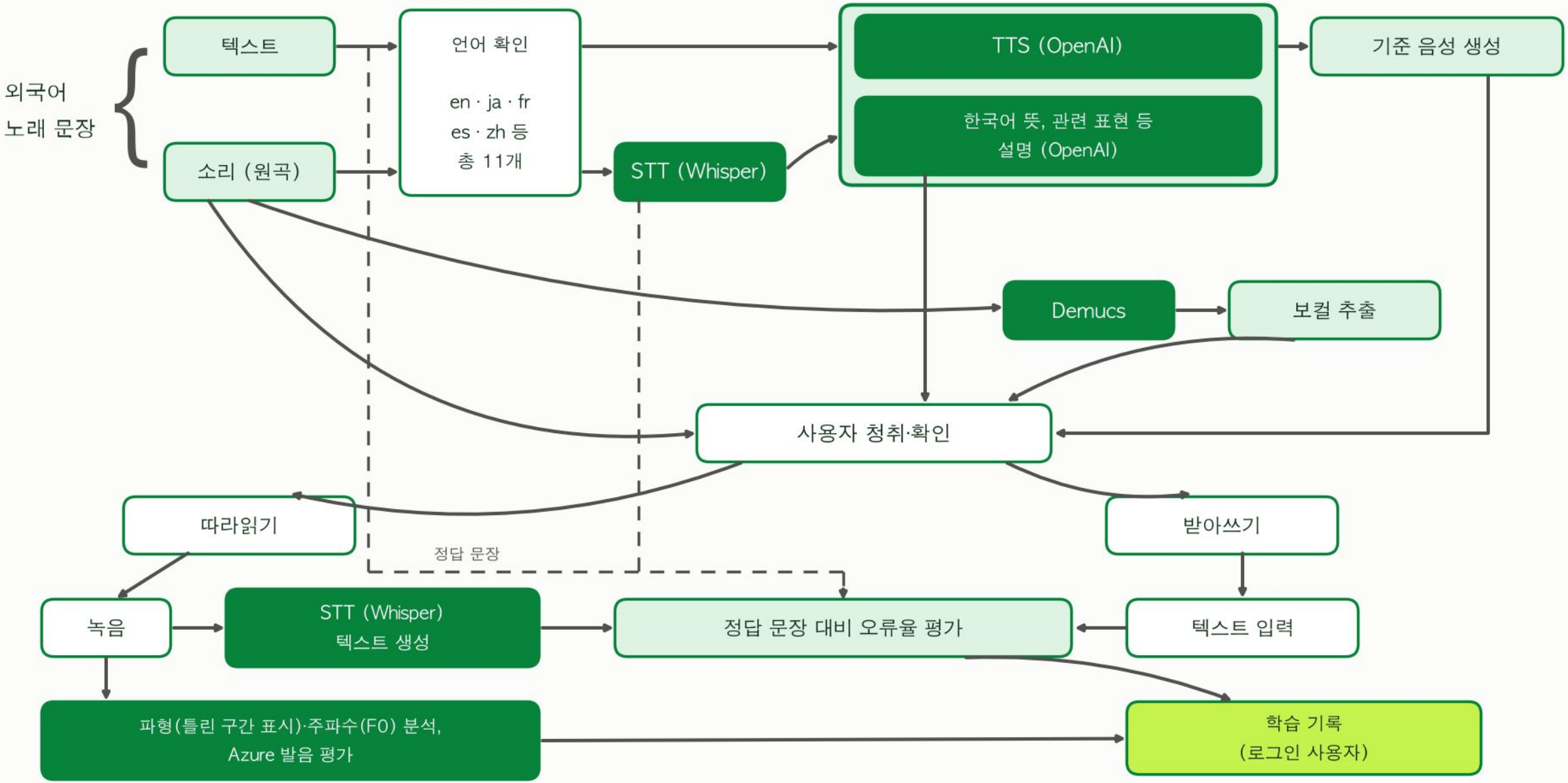
내가 이 프로젝트로 얻고 싶은 것

실제로 쓸 수 있는 외국어 학습 도구를 만들면서, 음성 데이터가 수집되고 처리되고 분석되어 사용자에게 전달되는 전체 과정을 이해하고 직접 다룰 수 있는 개발자로 성장한다

여섯 가지 목표

- **01** 쓰는 도구
- **02** 음성·음향 처리 내재화
- **03** AI 실험 능력
- **04** 소프트웨어 설계
- **05** 기술 → 사용자 경험
- **06** 과학적 호기심

프로그램 구조


외국어
노래 문장
텍스트
소리 (원곡)
언어 확인
en · ja · fr
es · zh 등
총 11개
STT (Whisper)
TTS (OpenAI)
한국어 뜻, 관련 표현 등
설명 (OpenAI)
기준 음성 생성
Demucs
보컬 추출
사용자 청취·확인
따라읽기
받아쓰기
정답 문장
녹음
STT (Whisper)
텍스트 생성
정답 문장 대비 오류율 평가
텍스트 입력
파형(틀린 구간 표시)·주파수(F0) 분석,
Azure 발음 평가
학습 기록
(로그인 사용자)

기술 스택

단계	기술	어디서
기준 음성	OpenAI TTS (gpt-4o-mini-tts)	OpenAI
인식	faster-whisper (문장 tiny · 노래 medium)	내 서버 CPU
채점	WER/CER · 편집 거리	내 서버
소리 그림·틀린 구간	STFT · Log-Mel · FFT, 단어별 시간	내 서버
목소리 높낮이	pYIN (F0)	내 서버
보컬 분리	Demucs htdemucs	내 서버
발음 평가	Azure Speech	Azure
저장·배포	PostgreSQL · Oracle Cloud VM · HTTPS · 로그인	내 서버

수업 내용의 적용

017_sound 수업의 코드 흐름을 그대로 따르고, 실제 소리·노래에 맞게 응용한 곳만 덧붙였다

02 리샘플링 → audio.to_16k_mono	g = gcd(sr, orig_sr) # 44100→16000이면 (160, 441) x = resample_poly(x, sr // g, orig_sr // g) # 16 kHz 모노로 맞춤	수업과 같음
03·lesson04 STFT → visualisation.stft_power	hz, t, spec = stft(sound, fs=sr, window="hann", nperseg=512, noverlap=512-128) power = abs(spec) ** 2 # 창 512, 이동 128, 파워 = |X|²	수업과 같음 (인자를 한 줄씩 대조)

lesson04 Log-Mel
→ visualisation.logmel
	mel_power, _ = apply_mel_filters(hz, power, sr, 40) # Mel 필터 40개로 묶기
	log_mel = to_db(mel_power) # 로그(dB)로 바꿈
		수업과 같음 + 두 소리를 같은 색 범위로 비교

07 Whisper 최소 예제 → stt.load_model	WhisperModel(size, device="cpu", compute_type="int8") # 7강과 같은 설정 segments, _ = model.transcribe(audio, beam_size=1) # 후보 하나만 탐색	수업과 같음 + 모델 크기·VAD·온도 옵션
05 WAV와 정답 연결 → sentences.csv ↔ 녹음	sentences.csv 의 id=1 ↔ data/recordings/s01_take1.wav 녹음 파일 번호로 정답 문장을 찾아 인식 결과와 비교	같은 발상 〔추정〕 (5강 manifest의 id·file·reference)

STFT 창 512·이동 128은 수업 설정을 그대로 썼다. 앱에 덜 응용된 것: 창 길이·Mel 개수를 바꿔 보는 실험(노트북·이슈 #35에서만).

입력 '언어 알아보기' 기능

텍스트 ("내 문장")

직접 만든 파이썬 규칙 — AI 모델 아님

- 글자 종류(가나·한글·한자·키릴·라틴)를 세어 줄마다 제안
- 라틴 문자 줄은 자주 쓰는 낱말·특징 글자 점수로 영어·프랑스어 등을 구분
- 즉시 · 모델 호출·과금 없음
- 한계: 짧은 줄은 틀릴 수 있고, 한자만 있으면 일본어·중국어를 구분하지 못해 직접 선택 (중국어는 자동 제안 안 됨)

소리 ("내 노래")

Whisper 모델의 언어 감지

- 구간 앞 30초의 소리를 듣고 언어별 확률 계산 (글로 받아 적지 않음)
- STT보다 먼저 · 원곡 구간 · 앱이 지원하는 11개 안에서 다시 순위
- 처음 한 번은 모델을 읽느라 몇십 초
- 한계: 반주가 섞인 소리·섞인 언어 가사에서 흔들릴 수 있음 (추정)

둘 다 제안일 뿐 → 사용자가 '네 / 아니오 / 모르겠어요'로 확인 → 인식(STT) · 기준 음성(TTS) · 채점 · Azure 로캘에 사용

입력 유형에 맞는 판별 규칙을 적용하고 , 자동 감지는 후보를 좁혀 제안한 뒤 사용자가 최종 확인한다 .

11 개 언어의 구성

03 문자 체계로 제안

- ja 일본어
- ko 한국어
- ru 러시아어

가나 · 한글 · 키릴 문자

07 라틴 문자 규칙

- en · fr · es
- de · it · pt · sv

빈출 낱말 + 언어별 특징 문자

01 텍스트 직접 선택

- zh 중국어

한자만으로 일 · 중 구분이 어려움

공통 목록 관리

LANGS → COMMON 화면 선택 목록 + Whisper 후보 재정렬

languages.py 한 곳에서 이름 · 언어 코드 · Azure 로캘을 함께 관리

Whisper 후보 제한 실험

14 곡 실험 — 영어 6 · 프랑스어 2 · 일본어 6

감지 후보를 en / ja / fr 로 제한

모델	제한 전	제한 후
tiny	12 / 14	**12 / 14**
base	11 / 14	**13 / 14**
medium	11 / 14	**13 / 14**

base · medium 11/14 → 13/14

해석 주의 · 감지 실험은 영어 · 일본어 · 프랑스어 3 개 언어의 14 곡 표본이다 . 11 개 전체 언어의 인식 성능을 검증한 결과는 아니다 .

근거 : languages.py (LANGS / COMMON / suggest_from_probs) · 프로젝트 검토 문서 16.12 · 16.14

‘따라읽기' 평가 방법

오류율 WER / CER

Whisper

인식된 문장이 기준 문장과 얼마나 다른가?

오류율 (낮을수록 좋음)
발음 점수가 아님

한계 문맥으로 틀린 발음도 맞게 적을 수 있고, 쉼·속도·억양은 못 잡는다

파형 분석

틀린 구간 표시 · 스펙트로그램

**어느 단어 구간이 다르게 인식됐고
소리 모양은 어떻게 다른가?**

점수 없음
눈으로 보는 관찰

한계 단어 구간은 모델이 추정한 대략의 구간 · 그림만으로 발음을 판정하지 않음

F0 목소리 높낮이

pYIN

음높이가 시간에 따라 어떻게 변하나?

점수 없음
관찰용

한계 TTS 억양이 정답이 아님 · 무성음·쉼에는 값이 없고 상대 음높이만 비교

Azure 발음 평가

외부 서비스 · 선택

기준 문장에 비추어 발음·유창성·운율은?

서비스가 주는 점수
버튼을 눌렀을 때만

한계 한 사람·한 문장·1회 시험 · 운율·음소는 영어 중심 (7장)

하나의 소리를 서로 다른 층위로 분석

Azure API를 활용한 발음 평가 및 피드백 구현

① 도입 배경	처음 기획한 오류율(WER/CER)은 인식된 글의 일치도라서 한계가 있다. ⓐ 문맥으로 고쳐 적혀 오류율 0인데 Azure는 explore를 68점으로 봤다(내 녹음 1건) ⓑ 끊어 읽어도 오류율은 0.250으로 낮다 → 언어 학습에는 '어디를 어떻게 고칠지' 피드백이 필요
② 서비스 개요	Azure AI Speech 발음 평가. 정확도·유창성·완성도·운율·종합, 단어별 점수와 오류 종류(발음·빠짐·더해짐), 음소 펼침. 평가 기준은 TTS가 아니라 Azure의 원어민 발음 모델
③ 사용 방법	따라읽기 채점 후 'Azure로 평가하기' 버튼(누를 때만) → 녹음·기준 문장 글·언어만 전송(TTS 소리는 보내지 않음, 녹음은 서버에 저장 안 함) → 점수 카드 + 단어별 점수. 로그인·저장을 켜면 점수와 단어별 결과가 기록에 저장(녹음 제외)
④ 결과 예시	같은 문장을 네 가지로 읽은 비교표 (다음 장)
⑤ 한계	한 사람·한 문장·1회. 정상 읽기도 we'll 51·together 43으로 이미 낮아 완벽한 기준선이 아님. 운율·음절·음소는 영어 중심이고, 일본어·프랑스어는 기본 점수가 나오는 것까지만 확인

Azure 발음 평가 예시

문장: We'll sing together until the morning comes. (직접 쓴 문장, 11-05 화면 캡처 4장)

읽기 조건	정확도	유창성	완성도	운율	종합	Azure가 표시한 단어	오류율(WER)
① 정상	81	90	71	81	79	we'll 51 · together 43(발음), 나머지 91~97	(캡처 없음)
② 단어 생략	32	77	14	69	41	together·the·comes 빠짐	0.750
③ 끊어 읽기	45	49	14	66	38	We'll 빠짐, the 더해짐	0.250
④ 틀린 발음	56	88	43	74	61	We'll 빠짐, sing 47 · together 44 · the 55(발음)	0.375

- 정상(79)보다 세 경우 모두 종합이 내려갔다(41·38·61): 차이가 평가에 반영된다
- 끊어 읽기는 유창성이 90 → 49로 가장 크게 내려갔는데 오류율은 0.250으로 낮다 → 쉼·속도는 오류율이 못 잡고 Azure(유창성)가 잡는다
- 단어 생략은 '빠짐' 표시와 완성도 71 → 14로 반응했다

완성도·정확도 정의는 Microsoft 문서 기준[일반 지식]. 완성도 계산식은 문서에 없어 이 표본으로 확인하지 못했다 [미확인]

문장 인식 일치도 · Whisper 발음 점수가 아니라 문장 인식 오류율이에요

단어 오류율 · WER	글자 오류율 · CER	오류 종류
33.3%	**20.0%**	**1 · 1 · 0**
6단어 중 2개가 달라요	25글자 중 5개 (공백 제외)	바꿈 · 빠짐 · 더해짐

둘 다 오류율이라 낮을수록 좋아요. We'll과 We will은 같은 것으로 봐요.

녹음을 글로 인식한 뒤 기준 문장과 비교한 **오류율**이에요(낮을수록 좋아요). 발음 정확도 자체를 평가하는 점수는 아니에요. Whisper가 문맥으로 잘못된 발음을 고쳐 인식했을 수도 있고, 주변 소리가 크면 오류율이 올라갈 수 있어요.

단어 대조 · 위는 기준 문장, 아래는 인식한 내 녹음 · 보라 물결 밑줄은 다른 단어

i	want	to	explore	the	universe
—	1	to	explore	the	universe
빠짐	바꿈				

인식한 문장: One to explore the universe. · 인식에 0.7초 걸림

다시 듣기

표준 발음

0:00 / 0:03

내 녹음

0:00 / 0:06

Azure로 평가하기

정확도	유창성	완성도	운율	종합
46	72	33	72	51
음소가 원어민 발음과 얼마나 가까운가	단어 사이 쉼이 자연스러운가	기준 문장 중 발음된 단어의 비율	억양·강세·속도·리듬(영어만)	위 항목을 합친 참고 점수

› 점수는 어떻게 계산되나요? · Azure 공식 문서

단어별 정확도 · 보라 물결 밑줄은 점수가 낮거나 오류가 있는 단어, 연한 라임은 보통

I	want	to	explore	the	universe
빠짐	85	빠짐	51 발음	41 발음	97

⌄ 음소별 점수 보기 (점수가 낮은 단어)

음소는 IPA 기호예요. '(ə로 들림)'은 기대한 음소 대신 가장 가깝게 들린 음소예요.

explore: ɪ 100 · k 100 · s 100 · p 100 · l 100 · ɔɹ 45
the: ð 40 (t로 들림) · i 42

오류율 채점 알고리즘

정답과 인식 결과가 얼마나 다른지를, 작은 문제의 답을 표에 적어 가며 큰 문제의 답을 구한다

정답 (ref) \ 인식 결과 (hyp) →	∅	we	seeing	together
∅	0	1	2	3
we	1	0	1	2
sing	2	1	1	2
together	3	2	2	1
until	4	3	3	2

되짚은 길 (초록 칸):

we = we	일치
sing → seeing	바꿈(S)
together = together	일치
until → (없음)	빠짐(D)

한 칸을 채우는 규칙 (점화식)

d[i][j] = 세 값 중 가장 작은 것

· 위 칸 + 1 : 삭제 (정답 단어를 지움)

· 왼쪽 칸 + 1 : 삽입 (인식에만 있는 단어)

· 대각선 칸 + (같으면 0, 다르면 1) : 일치 · 바꿈

표의 오른쪽 아래 칸 = 최소 수정 횟수 = 2

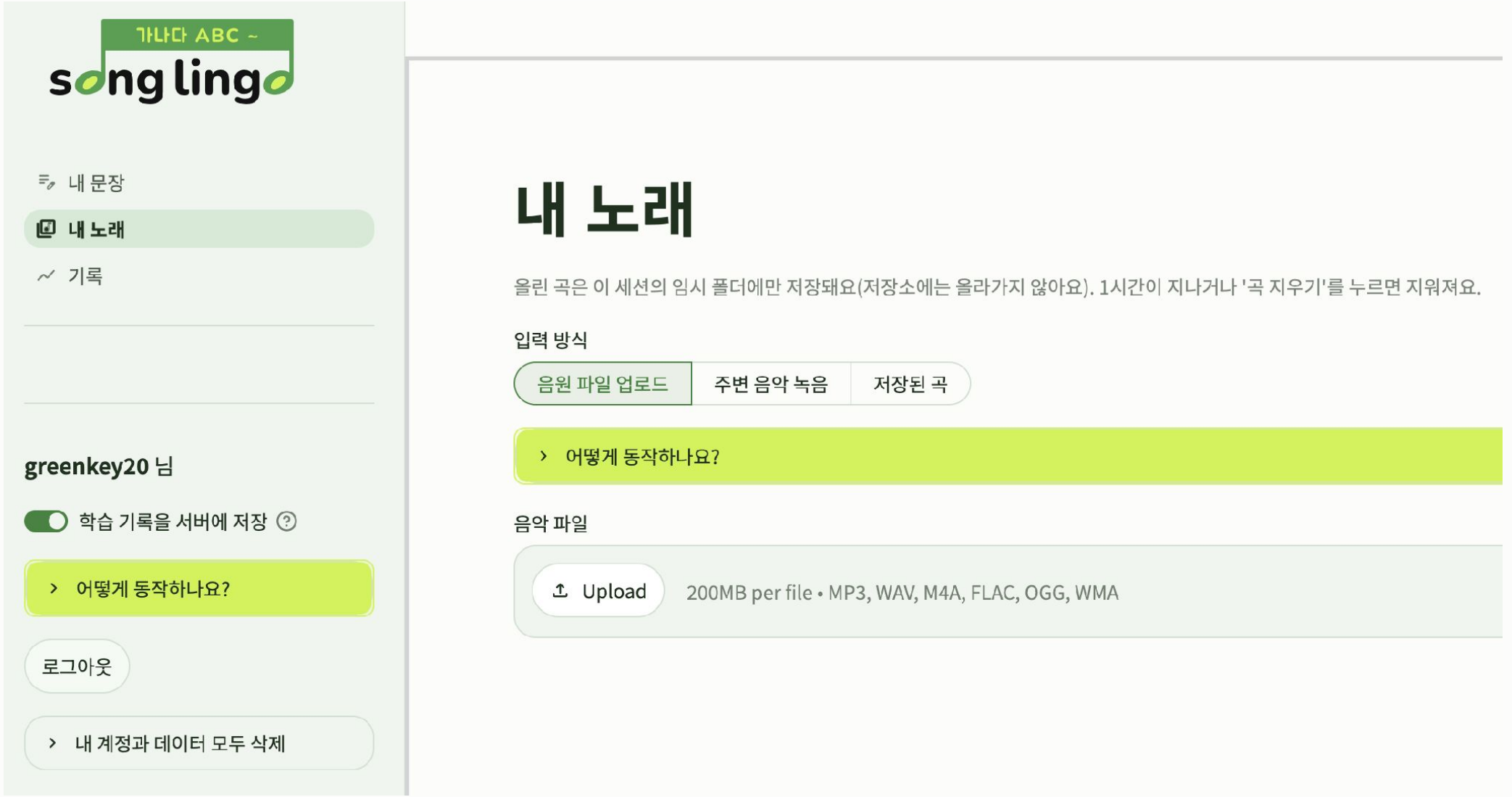
WER = (바꿈 S + 빠짐 D + 더해짐 I) ÷ 정답 단어 수 N = (1 + 1 + 0) ÷ 4 = 0.50

```
# scoring.edit_matrix 의 핵심 (내 코드)
cost = 0 if ref[i-1] == hyp[j-1] else 1     # 같으면 0, 다르면 1
d[i][j] = min(d[i-1][j] + 1,                # 삭제
              d[i][j-1] + 1,                # 삽입
              d[i-1][j-1] + cost)           # 일치 또는 바꿈
```

이해한 방법: 점화식 → 표를 한 칸씩 넘기는 도구(DP_편집거리_이해하기.html)

동률일 때 비슷한 단어끼리 짝짓는 규칙은 이 프로젝트에서 덧붙인 부분(수업에 없음)

프로그램 시연


가나다 ABC ~
song lingo
내 문장
내 노래
기록
greenkey20 님
학습 기록을 서버에 저장
어떻게 동작하나요?
로그아웃
내 계정과 데이터 모두 삭제
내 노래
올린 곡은 이 세션의 임시 폴더에만 저장돼요(저장소에는 올라가지 않아요). 1시간이 지나거나 '곡 지우기'를 누르면 지워져요.
입력 방식
음원 파일 업로드
주변 음악 녹음
저장된 곡
어떻게 동작하나요?
음악 파일
Upload
200MB per file • MP3, WAV, M4A, FLAC, OGG, WMA

딥러닝 모델 1_Whisper

합성곱 2개 + Transformer 인코더–디코더 · 입력은 항상 30초 창(80 × 3000) · LSTM 0개

① 입력 소리 (직접 쓴 문장의 TTS, 3.2초) — 30초 창의 앞쪽에 놓음



② Log-Mel (80 × 3000) — ①과 같은 가로축(시간)

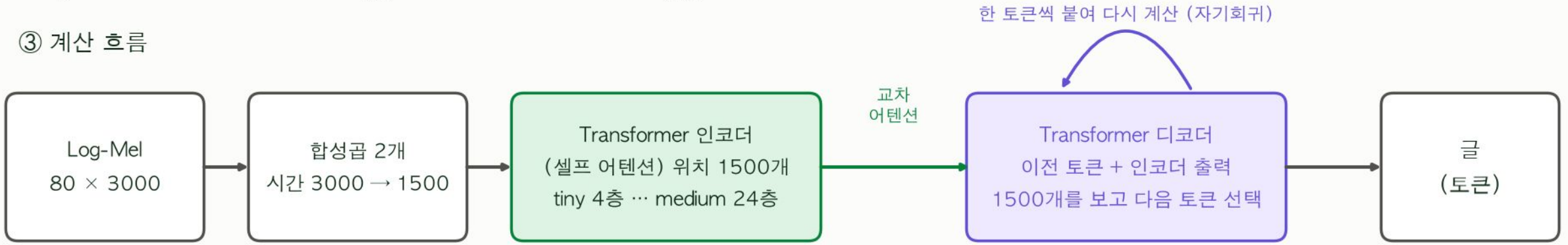


확인한 것 〔model.bin을 직접 읽음〕

- 층 수 4 → 6 → 12 → 24, 차원 384 → 1024
- LSTM·GRU 변수 0개 (Transformer만, 입력 앞 합성곱 2개)
- tiny는 파라미터의 절반 이상이 '단어 사전'
- 입력은 항상 (80, 3000) — 짧아도 줄지 않음
  (전체 시간은 생성 토큰 수에도 좌우됨)

못 본 것: 가중치 값, 어텐션 패턴

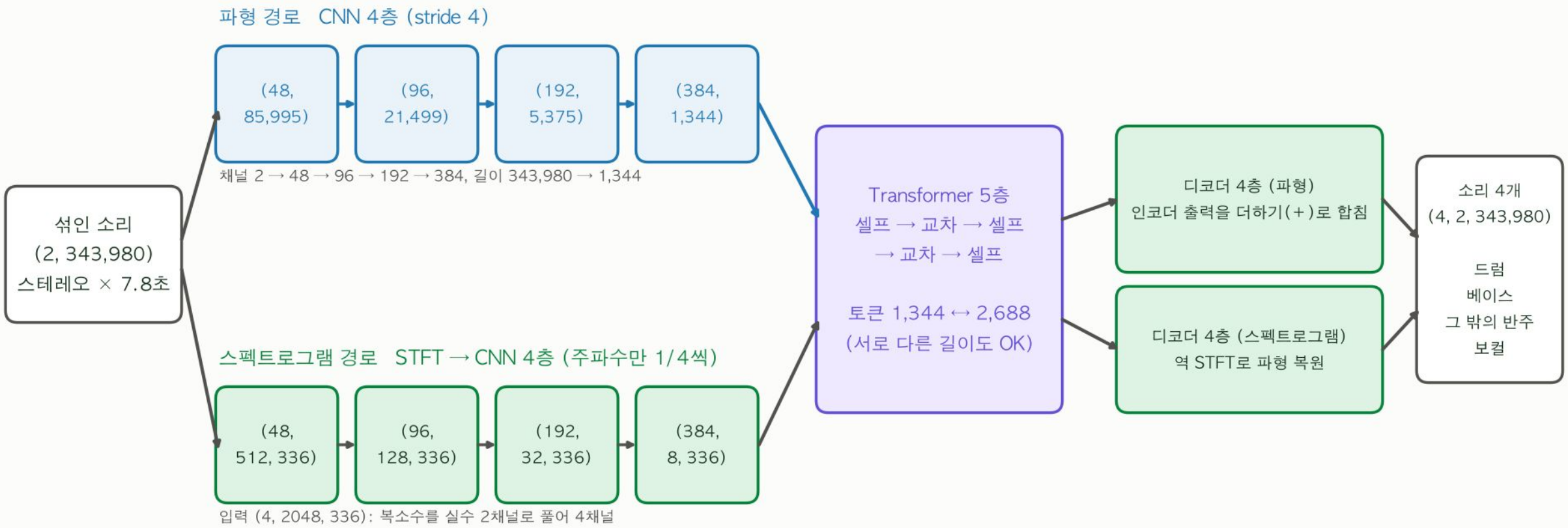
③ 계산 흐름



근거: experiments/model anatomy/whisper_anatomy_claude_code.ipynb (4·5·9절) · 소리 예시는 직접 쓴 문장의 OpenAI TTS · 개념도이며 실제 내부 층은 더 복잡

딥러닝 모델 2_Demucs

같은 소리를 파형·스펙트로그램 두 경로로 보고 → CNN 압축 → Transformer 결합 → 디코더 복원 (7.8초 조각)



기억할 네 문장: ① 두 표현(파형·스펙트로그램) ② CNN이 경로마다 압축해 특징 벡터 생성 ③ Transformer가 경로 안과 경로 사이 정보 결합 ④ 디코더가 소리로 복원

파라미터 4,198만 개 중 75%가 Transformer — 학습되는 숫자의 '개수 비중'이며 연산량·성능 기여도가 아님 · nn.LSTM 0개 · 못 본 것: 어텐션 가중치, 각 경로가 배운 것

근거: experiments/model anatomy/demucs/demucs_structure_analysis_…ipynb (4·7·8절, 실행해 기록한 값) · 사전학습 모델의 추론과 구조 분석이며 직접 학습한 것이 아님 · 개념도

두 가지 모델 비교

둘 다 CNN과 Transformer를 쓰지만, 출력이 다르면 놓는 자리가 달라진다



공통점: 둘 다 CNN과 어텐션을 쓴다 — 다만 쓰는 자리가 다르다 (Whisper: Transformer 인코더·디코더, HTDemucs: CNN 인코더·디코더 + 가운데 Transformer)

차이점: Transformer 디코더가 글을 한 토큰씩 생성 ↔ CNN 디코더가 소리를 복원 (HTDemucs에는 Transformer 디코더가 없음) · 개념 비교용 그림

시행착오·실험 1_노래와 가사를 어떻게 확보할까?

초기 구상: 노래 검색 → 곡 정보 확인 → 가사 확보 → 학습 문장 선택

접근 방식	처리 흐름	고려한 문제
① 곡 식별 후 가사 검색	주변 소리 → Shazam류 음원 식별 API → 곡 정보 → 가사 제공 API	API 키·비용·이용 조건, 가사 제공 범위 및 저작권
② 음원 직접 입력 후 STT	파일 업로드 / 주변 음악 녹음 → Whisper STT → 인식 결과 수정	반주·발성·음질에 따른 인식 정확도
③ 가사 텍스트 직접 입력	사용자가 문장·가사 입력 → 표준 발음·채점	자동 곡 검색은 없지만 가장 단순한 대안

- 가사 사이트 스크래핑은 이용약관·저작권·유지보수 위험으로 제외
- 음원 식별 API·가사 제공 서비스는 키·약관·요금·언어 지원이 불확실해 보류
- 파일 업로드와 마이크 녹음을 같은 STT 파이프라인으로 통합

현재 흐름: 음원 업로드 / 주변 음악 녹음 → 로컬 Whisper STT → 사용자 수정 → 따라읽기·받아쓰기

질문 1: 노래 구간 STT — 분리한 보컬이 항상 더 잘 인식될까?

실험 질문

원곡(mix)과 Demucs 분리 보컬(vocals) 중 Whisper 입력으로 어느 쪽이 적합한가? VAD·온도 폴백·모델 크기의 영향도 확인

데이터와 비교 방법

- 영어 노래 20초 구간: 8개(231단어) + 다른 곡 6개(165단어) = 14개, 396단어
- 같은 구간의 원곡·분리 보컬을 각각 준비
- 정답은 분리 보컬이 아닌 원곡을 들으며 직접 작성
- 지표: micro-WER (낮을수록 좋음)

단계	조건	결과
A 진단	temperature=0.0 (폴백 끔)	반복 루프 많음. base+보컬+VAD 켬 0.494가 최고로 보임
B 기본 재검증	폴백 켬, 3회 반복	원곡+VAD 켬은 5/8 구간 빈 결과 → VAD 끄기. 최고 base+보컬+VAD 끔 0.382
C 모델 크기	VAD 끔	보컬 base 0.382 → small 0.326 → medium 0.257. 처음 8개: medium 원곡 0.177 / 보컬 0.257
D 새 곡 6개	medium	원곡 0.303 / 보컬 0.285. 14개 합산 원곡 0.230 / 보컬 0.269 → 우열 미확정

결정: medium + 원곡 + VAD 끔 + 기본 폴백. Demucs 분리는 필수 전처리가 아니라 듣기 연습 선택 기능

한계: 영어 노래 14개 구간, 정답 작성자 1명. 3회 반복은 통계적 유의성 검정이 아님

질문 2: 보컬 분리 효과는 입력의 음악적 특성에 따라 달라졌을까?

서로 다른 음악적 특성의 구간에서 원곡(mix)과 분리 보컬(vocals) 비교 (WER 낮을수록 좋음, 원곡 → 보컬)

유형 (표본 수)	관찰 결과	해석
또렷한 솔로 보컬 (5)	c03·c09·c10은 분리 보컬 WER이 낮음, c01·c02는 차이 거의 없음	분리가 항상 필요한 것은 아니며 구간별 차이가 있음
두꺼운 반주 (4)	c04 0.41→0.59, c05 0.34→0.79로 원곡 우세. c11 동일 (0.09)	반주가 두꺼워도 분리가 항상 개선하지는 않음
합창·아카펠라 (4)	c07 차이 거의 없음, c13 동일(0.06), c14 보컬 우세(0.29 →0.19)	보컬 편성만으로 우열을 설명하기 어려움
힙합·랩 (1)	c12는 원곡·보컬 모두 0.6 이상. 이 구간을 빼면 검증 결과의 우열이 바뀜	어려운 단일 구간이 전체 결과에 미치는 영향 확인

- 한쪽이 모든 유형에서 일관되게 우세하지는 않았다
- 음악적 특성에 따라 분리 효과가 달라질 가능성을 관찰했다

탐색적 분석이며 유형별 성능 순위나 일반화된 결론이 아니다. 유형별 표본 수가 제한적이다.

실험 결과 및 ToDo

이번 실험으로 확인한 것

- 이번 표본에서 medium은 base보다 나은 인식 결과
- 원곡에 VAD를 적용하면 특정 구간 가사가 통째로 사라지는 문제 재현
- 원곡과 분리 보컬의 우열은 구간에 따라 달라짐

의미: 입력이 항상 정답이라고 증명한 것이 아니라, 가설을 실험 결과에 맞춰 수정하고 후속 검증 질문을 구체화한 것

현재 실험의 한계	다음에 시도할 것
표본 수와 유형 불균형: 20초 구간 14개, 힙합은 1개뿐	유형별로 서로 다른 곡 10~15개씩 추가 확보
유형 분류 기준의 혼재: 장르·반주 밀도·보컬 편성·발성 방식이 섞임	반주 밀도 / 보컬 구성 / 발성 스타일을 독립 속성으로 기록
정답 데이터의 불확실성: 한 사람이 원곡을 듣고 작성	2인 이상 독립 전사·교차 검수, 채점 규칙 사전 정의
평가와 반복의 한계: 3회 반복은 유의성 검정이 아니며 WER은 발음 평가가 아님	곡별·유형별 WER, 오류 유형(S/D/I), 빈 인식, 반복 현상 평가
분리 품질의 별도 측정 부족	청취 평가, 정답 stem이 있는 데이터에서 SDR 등 확인

시행착오·실험 2_AI가 작성한 코드를 어떻게 다룰 것인가?

코드 생성보다 어려웠던 것은 이해와 검증이었다

① 문제 발견

19~34%

기존 주요 모듈의 주석·docstring 비율

수업에서 배운 코드와 AI가 추가한 코드의 경계가 불분명했다.

↓

② 개선 시도

- 파일·함수의 역할과 입출력 명시
- 수업과 같은 부분 / 수업에서 다루지 않은 부분 구분
- 인라인 주석은 코드의 '왜'를 설명
- 주석 수정 시 기존 동작 유지

↓

③ 검증

동일

주석 제외 AST 비교

65개

당시 단위 테스트 통과

주석만 바뀌었고 실행 코드는 바뀌지 않았는지 확인했다.

④ 주석을 읽을 수 있는 것과 코드를 이해하는 것은 다르다. 아직 내 말로 설명 못 하는 부분은 핵심 함수부터 직접 설명하며 공부할 계획

프로젝트의 결과 및 ToDo

실사용 가능한 학습 도구에서 시작해, 음성 데이터의 이해와 활용으로

RESULT · 구현 결과

- Oracle Cloud 배포·HTTPS
- 아이폰 홈 화면 바로가기
- 로그인·PostgreSQL 사용자별 저장
- STT·TTS·채점·음향 분석 통합
- 비밀번호는 scrypt 해시, 음성 파일은 DB 미저장

LIMIT · 검증 한계

- 다국어별 품질 검증 부족
- 성조·악센트·억양 평가 미지원
- 1인·조용한 환경 중심 테스트
- 학습 기록·복습 기능의 확장 필요

NEXT · 다음 단계

- 다양한 화자·언어 테스트
- 학습 이력·복습 기능 고도화
- 음향 기반 시각화·창작
- GPU 기반 Demucs 학습 탐구

소리가 데이터가 되고, 모델을 지나, 학습 경험으로 돌아오는 과정을 구현하고 탐구한 프로젝트